# Cadenas y Strings en C++

METODOLOGÍA DE LA PROGRAMACIÓN

GRADO EN INGENIERÍA INFORMÁTICA

DECSAI

UNIVERSITAS GRANATENSIS · 1531 · PLUS ULTRA

# Índice

- Introducción.
- C-string.
- La clase string.

# Introducción

- C++ permite representar cadenas de varias maneras.
- Cada forma tiene sus pros y sus contras y requiere de diferentes funciones para su manejo.
- Existen funciones que convierten de una representación a otra.
- Principalmente son 2:
  - c-string.
  - Clase string.

# C-string

- Una cadena c-string es una cadena de caracteres (char *) que tiene la siguiente forma:

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| h | o | l | a | \0 | ? | ? | ? | ? | ? |

- **Consideraciones**:
  - Aunque pueda haber más espacio reservado, el tamaño del c-string se cuenta hasta que aparece el carácter delimitador \0. Por tanto, la cadena de arriba tiene tamaño 4 aunque haya 10 posiciones reservadas.
  - Si usamos un char * sin que el delimitador \0 esté presente al final, entonces tenemos una cadena de caracteres pero no un c-string.
  - Se puede utilizar tanto con memoria estática como con memoria dinámica.

# C-string

- Todo lo que hay después del primer \0 encontrado, no cuenta como perteneciente al c-string.

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| h | o | l | a | \0 | a | b | \0 | ? | ? |

Las funciones de tratamiento de c-string consideran que el c-string es “hola”.

- Es importante que haya suficiente espacio reservado para almacenar la información. Si no estaremos produciendo una violación de segmento.

- El conjunto de funciones que trabajan sobre c-string pertenecen a una biblioteca de funciones y no conforman una clase.

# C-string

## Ejemplos de declaración con memoria estática:

```
//con memoria estática
char A[7] = {'V','a','l','i','d','o','\0'};
char B[] = {'V','a','l','i','d','o','\0'};
char C[] = "Valido"; //El compilador añade '\0' al final automaticamente
char D[3] = {'M','a','l'}; //es valido pero no es un c-string
char F[100] = "Valido"; //sobra espacio pero no pasa nada.
```

## Ejemplos de declaración con memoria dinámica:

```
//con memoria dinámica
char * E = new char[5];
E[0] = 'b';
E[1] = 'i';
E[2] = 'e';
E[3] = 'n';
E[4] = '\0';
```

No podemos utilizar el inicializador ☹
pero si podemos usar algunas de las funciones de la biblioteca de c-string

# C-string

## Literales de cadena

- Cuando declaramos un c-string como const char *, entonces lo llamamos literal.
- Los literales, al ser const, no se pueden modificar internamente.
- Es buena práctica utilizar literales para aquellas cadenas que no pensemos modificar.

```
//literal
const char * lit = "Literal";
//lit[2] = 4; //Error de compilación
```

# C-string

## Biblioteca c-string

- Es una colección de funciones que permiten el manejo de cadenas c-string.
- No conforman una clase por lo que debemos añadir la biblioteca y utilizarlas como funciones normales cuyos parámetros serán c-strings, no como métodos de una clase.
- Las más utilizadas son:
  - strcpy
  - strlen
  - strcat
  - Más funciones: http://www.cplusplus.com/reference/cstring/

Hay que poner #include<cstring> para usarlas

# C-string

## Función strcpy

- Copia un c-string en otro. El contenido inicial del c-string destino es eliminado.

```
char * strcpy (char * destination, const char * source);
```

- Ejemplo de uso:

```
//Ejemplo de uso de strcpy
char dest[] = "Hola a todos";
char orig[] = "Tardes";
cout << orig << " " << dest << endl;
strcpy(dest,orig);
cout << orig << " " << dest << endl;
```

```
Tardes Hola a todos
Tardes Tardes

Process returned 0 (0x0)   execution time : 1.350 s
Press any key to continue.
```

# C-string

## Función strlen

- Devuelve el tamaño de un c-string.
- En el tamaño, no cuenta el carácter '\0'.
- Ejemplo:

int * strlen(const char * str);

```
char dest[] = "Hola a todos";
char orig[] = "Tardes";
cout << strlen(dest) << " " << strlen(orig) << endl;
int tamanio = strlen(orig);
```

```
12 6

Process returned 0 (0x0)   execution time : 1.279 s
Press any key to continue.
```

# C-string

## Función strcat

- Concatena una cadena a otra. La cadena de destino queda modificada.
- La función modifica el espacio reservado en destination para que la concatenación quepa.
- Esta función, al igual que las anteriores, funciona tanto con memoria estática como con memoria dinámica.

```
char * strcat ( char * destination, const char * source);
```

# C-string

## Función strcat

- Ejemplos:

```
char origen[] = "todos";
char destino[] = "Hola a ";
strcat(destino,origen);
cout << destino << endl;
cout << strlen(destino) << endl;

char * destdin = new char [20]; //Debe reservarse memoria antes de usar
                                //strcpy o strcat
strcpy(destdin,"Hola ");
char * origdin = new char [20];
strcpy (origdin,"a todos");
strcat(destdin,origdin);
cout << destdin << endl;
```

```
Hola a todos
12
Hola a todos

Process returned 0 (0x0)   execution time : 1.254 s
Press any key to continue.
```

# La clase string

## Introducción

- La clase string permite el manejo de cadenas.
- Al ser una clase, declaramos objetos de tipo string y manejamos los objetos a partir de los métodos implementados en la clase.
- Para poder usar la clase string debemos incluir: #include<string>
- string es una cadena de longitud variable que utiliza memoria dinámica en su implementación interna.
- Más información de la clase y métodos: https://www.cplusplus.com/reference/string/string/

# La clase string

## Representación interna de la clase string

- Internamente, la clase string utiliza los siguientes datos miembros en su representación:

| Datos miembro de la clase string |
| --- |
| char * datos; |
| Char * fin; |
| int tam; |

Indica el comienzo de la cadena

Apunta al carácter \0 de la cadena

Tamaño actual de la cadena

# La clase string

## Constructores de la clase string

- **Constructor por defecto**: string a;
- **Constructor con repetición**: string b(6,'a');

```
aaaaaa

Process returned 0 (0x0)   execution time : 1.424 s
Press any key to continue.
```

- **Constructor a partir de char ***:

```
const char * lit = "Hola";
string c(lit);
cout << c << endl;
```

```
Hola

Process returned 0 (0x0)   execution time : 1.271 s
Press any key to continue.
```

# La clase string

## Constructor de copia

- El constructor de copia de la clase string crea un nuevo string a partir de otro string.
- La implementación del constructor crea 2 objetos separados e independientes, tal y como debe ser. No utiliza el de por defecto.

```
string d = "Hola a todos";
string e(d);
cout << e << " " << d << endl;
```

```
Hola a todos Hola a todos

Process returned 0 (0x0)   execution time : 1.500 s
Press any key to continue.
```

# La clase string

DECSAI

## Consultores de los datos miembro de la clase string

- Funciones que permiten consultar los datos de la clase sin modificarlos.
- **size y length**: Devuelven el tamaño del string sin contar el carácter \0.
- **Data y c_str**: Devuelve un const char * con los datos de la cadena que representa el objeto string.
- **find**: Está sobrecargada. Encuentra el carácter o la subcadena y devuelve la posición en la que comienza. Se le puede indicar como segundo parámetro en qué posición debe comenzar la búsqueda.

# La clase string

## Ejemplos de uso:

```
string prueba = "Hola";
cout << prueba.size() << " " << prueba.capacity() << endl;
cout << prueba.length() << endl;

//char * cdeprueba = prueba.data(); //ERROR: data y c_str devuelven un const char
const char * cdeprueba = prueba.data();
const char * cdeprueba2 = prueba.c_str();

prueba = "Hola a todos";
cout << prueba.find('a') << endl;
cout << prueba.find('a',4) << endl;
cout << prueba.find("todos") << endl;
```

# La clase string

DECSAI

## Modificadores de la clase string

- Funciones que modifican los objetos miembro de la clase string.
- Algunos ejemplos:
  - **clear**: Borra todo el contenido de una cadena. Los punteros a la cadena pueden invalidarse.
  - **push_back**: Añade un elemento al final de la cadena.
  - **resize**: Modifica el tamaño de la cadena rellenando los huecos con un carácter.
  - **Operador +**: Concatenación de strings.

# La clase string

DECSAI

## Ejemplos de uso:

```
string cad = "Hola";
cad.push_back('a');

string cad2 = "Hola";
cad2 = cad2 + " a todos";

string cad3 = "Esto es una cadena";
cad3.clear();

string cad4 = "Esto es una cadena";
string cad5 = cad4;
cad4.resize(5);
cad5.resize(21,'.');
cout << "Cad1: " << cad << endl;
cout << "Cad2: " << cad2 << endl;
cout << "Cad3: " << cad3 << endl;
cout << "Cad4: " << cad4 << endl;
cout << "Cad5: " << cad5 << endl;
```

```
Cad1: Holaa
Cad2: Hola a todos
Cad3:
Cad4: Esto
Cad5: Esto es una cadena...

Process returned 0 (0x0)   execution time : 1.416 s
Press any key to continue.
```

# La clase string

## Más modificadores: erase

```
string& erase (size_t pos = 0, size_t len = npos);
```

- Elimina una cantidad de caracteres específica a partir de una posición.
- Tiene 2 parámetros:
  - **Primer parámetro**: a partir de qué carácter empezamos a borrar.
  - **Segundo parámetro**: Cantidad de caracteres a eliminar.
- Ejemplo:

```
string ejemploborrado = "Esto es una prueba";
ejemploborrado.erase(2,8);
cout << ejemploborrado << endl;
```

Salida = Esa prueba.

# La clase string

## Más modificadores: replace

- Tiene 3 parámetros:
  - Posición a partir de la cual reemplazo.
  - Caracteres que elimino a partir de esa posición.
  - Qué elementos añado.

- Ejemplo:

```
string cadena = "Holaaaaa";
cadena.replace(4,3," a todos");
cout << cadena << endl;
```

- Salida: Hola a todosa

# La clase string

## Concatenación de llamadas a métodos

- Los métodos de la cadena string se pueden concatenar:

```
string cadenat = "Holaaaaa";
cout << cadenat.replace(1,2,"mundo").replace(4,5,"todos").size() << endl;
```

- Resultado: 11

- Se aplica primero el primer método, luego el segundo, luego el tercero …

# La clase string

## Accediendo a los caracteres del string.

- 2 métodos:
  - Método at.
  - Operador corchete.
- Ambos métodos de acceso tienen opciones constantes y no constantes.

# La clase string

## Ejemplo de uso de los operadores de acceso

```
//FUNCION AT
string cadena = "Hola a todos";
//version de no const
cadena.at(4)='-';
cout << cadena << endl;
//version de const
const char primercaracter = cadena.at(0);

//Operador []
cadena[6] = '-';
const char segundocaracter = cadena[1];
cout << cadena << endl;
cout << "Caracteres: " << primercaracter << " " <<
segundocaracter;
```

```
Hola-a todos
Hola-a-todos
Caracteres: H o
Process returned 0 (0x0)   execution time : 1.274 s
Press any key to continue.
```